E.T. N.° 35 "Ing. Eduardo Latzina"
Alumno: Santino Crespo | Curso: 6.° 2.ª

Mini actividad – Refactoring + Tests Unitarios

Proyecto: TareasYA (basado en la SPEC del TP N.° 1)

Funcionalidad elegida

Alta de tarea (crear una tarea nueva): validación de los datos ingresados y creación del objeto tarea, dentro del componente `TaskForm`.

1. Análisis de la IA (antes de tocar código)

Sobre `TaskForm.before.jsx` se detectó:

• **Responsabilidades mezcladas:** handleSubmit valida datos, genera el id, arma el objeto tarea, guarda en localStorage y limpia el formulario, todo junto.

• **Función demasiado larga:** toda la lógica de negocio vive dentro del manejador de submit del componente.

• **Difícil de testear:** no se puede probar la validación ni la creación de la tarea sin renderizar el componente React completo.

• **Poco reutilizable:** si se quisiera crear tareas desde otro lugar (por ej. importación), habría que duplicar la lógica.

• No se detectó duplicación de código todavía (el proyecto es chico), pero sí una clara oportunidad de separar responsabilidades.

2. Mejora aplicada (una sola, sin cambiar el comportamiento)

Se extrajo la lógica de **validación** y **creación de la tarea** a un módulo separado y puro: `src/logic/taskLogic.js`, con las funciones `validarDatosTarea(datos)`, `crearTarea(datos)` y `generarIdTarea()`.

El componente `TaskForm.jsx` quedó reducido a manejar el estado del formulario y llamar a esas funciones. El comportamiento visible para el usuario (mensajes de error, datos guardados, limpieza del formulario) **no cambió**.

Código antes → después

```
// TaskForm.jsx (ANTES) - fragmento de handleSubmit
function handleSubmit(e) {
  e.preventDefault();

  if (!titulo || titulo.trim() === "") {
    setError("El título es obligatorio");
    return;
  }
```

```
  if (!materia || materia.trim() === "") {
    setError("La materia es obligatoria");
    return;
  }
  // ... más validaciones inline ...

  const id = Date.now().toString(36) + Math.random().toString(36).slice(2, 8);
  const nuevaTarea = {
    id, titulo: titulo.trim(), descripcion: descripcion.trim(),
    materia: materia.trim(), fechaLimite: fechaLimiteDate.toISOString(),
    prioridad, estado: "pendiente", favorito: false,
    fechaCreacion: new Date().toISOString(),
  };

  const tareasActuales = loadTasks();
  tareasActuales.push(nuevaTarea);
  saveTasks(tareasActuales);
  // limpieza de formulario...
}
```

```
// taskLogic.js (DESPUES) - logica pura extraida
export function validarDatosTarea(datos) {
  const { titulo, materia, fechaLimite, prioridad } = datos;
  if (!titulo || titulo.trim() === "") {
    return { valido: false, error: "El título es obligatorio" };
  }
  if (!materia || materia.trim() === "") {
    return { valido: false, error: "La materia es obligatoria" };
  }
  if (!fechaLimite || isNaN(new Date(fechaLimite).getTime())) {
    return { valido: false, error: "La fecha límite no es válida" };
  }
  if (!PRIORIDADES_VALIDAS.includes(prioridad)) {
    return { valido: false, error: "La prioridad no es válida" };
  }
  return { valido: true, error: null };
}

export function crearTarea(datos) {
  const { titulo, descripcion = "", materia, fechaLimite, prioridad } = datos;
  return {
    id: generarIdTarea(),
    titulo: titulo.trim(),
    descripcion: descripcion.trim(),
    materia: materia.trim(),
    fechaLimite: new Date(fechaLimite).toISOString(),
    prioridad, estado: "pendiente", favorito: false,
    fechaCreacion: new Date().toISOString(),
  };
}

// TaskForm.jsx (DESPUES) - handleSubmit queda delgado
function handleSubmit(e) {
  e.preventDefault();
  const datos = { titulo, descripcion, materia, fechaLimite, prioridad };
  const { valido, error } = validarDatosTarea(datos);
  if (!valido) { setError(error); return; }

  const nuevaTarea = crearTarea(datos);
  saveTasks([...loadTasks(), nuevaTarea]);
  limpiarFormulario();
}
```

3. Tests unitarios generados

Archivo: `taskLogic.test.js` (Vitest). Se probaron las dos funciones extraídas con casos correctos, inválidos y límite:

Test	Tipo	Qué valida
Datos completos y válidos	Correcto	validarDatosTarea acepta una tarea bien formada
Título vacío (solo espacios)	Inválido	Se rechaza y devuelve el mensaje de error correcto
Prioridad fuera de baja/media/alta	Inválido	Se rechaza una prioridad no permitida
Fecha límite no parseable (31-31-2026)	Límite	Una fecha con formato imposible se detecta como inválida
Título con espacios al borde	Límite	El trim() hace que un título con espacios sea válido
Creación con datos válidos	Correcto	crearTarea arma el objeto con estado pendiente, favorito false, id único y fecha ISO
Descripción vacía/omitida	Límite	No rompe la creación, queda como string vacío
Dos ids generados seguidos	Límite	generarIdTarea no repite valores en llamadas consecutivas

4. Resultado de la ejecución

Comando ejecutado: `npx vitest run`

```
✓ src/logic/taskLogic.test.js (8 tests) 8ms

  Test Files  1 passed (1)
       Tests  8 passed (8)
    Duration  251ms
```

Los 8 tests pasan correctamente después del refactoring, confirmando que el comportamiento de la funcionalidad de alta de tarea **no cambió**.